Blue Impulse — Stage 2 Video Guide V1.0

Collection: Pip's Shift — five connected tutorial videos
Team/module: Blue Impulse — Hardware, Networking and Behaviorist AI
Revision date: 7 November 2026

A fictional campus delivery rover faces a blocked route, a waiting processor, a late message, ambiguous commands and a small program whose result must be stored. Each ten-minute tutorial resolves one problem through an analogy, a precise mechanism, a worked demonstration and an active learning check. The story name is Pip; the packet demonstration uses device ID robot-7.

This single guide includes titles, filenames, durations, sizes, goals, prerequisites, section timestamps, connections to the six-chapter learning system, and every recorded narration cue with teaching directions and checked practice results. Submit the five MP4 files individually with this PDF. Supporting activity files may be supplied as a separate small ZIP; a combined video archive does not replace the individual MP4 submission.

Viewing order and file inventory

Order / title	Filename	Duration / size
01 — Basic reinforcement learning	BlueImpulse_Video01_Basic_Reinforcement_Learning.mp4	10:00 / 47.43 MB
02 — How hardware enables AI	BlueImpulse_Video02_How_Hardware_Enables_AI.mp4	10:00 / 48.16 MB
03 — Network challenges	BlueImpulse_Video03_Network_Challenges.mp4	10:00 / 48.90 MB
04 — Networking for behaviorist AI	BlueImpulse_Video04_Networking_for_Behaviorist_AI.mp4	10:00 / 50.06 MB
05 — Program execution	BlueImpulse_Video05_Program_Execution.mp4	10:00 / 45.85 MB

The numbered order begins with a motivating learning loop, then develops its hardware and network requirements before tracing instruction execution. Learners unfamiliar with CPUs may watch Video 05 first and return to the numbered order. Chapter references are given by chapter number, chapter name and exact subtopic title. The guide does not require pagination to remain fixed.

Access, captions and learner setup

All required explanations and practice inputs are in the delivered files. Use a media player with MP4 support. Captions are burned into the videos, and editable SRT/VTT files accompany the collection. No account or internet connection is needed for the paper exercises or local Python examples. Python 3.9 or newer is optional; the code uses the standard library. A calculator and paper are sufficient.

Media and production attribution

Neogpt/Codex assisted with the revised scripts, procedural graphics, sound design, documents and checking. Synthetic narration uses Kokoro ONNX, voice am_michael; it is not a student voice recording. Inter is the single type family, with deep blue #182445, vivid pink #FF4F8B and near-white #F4F7FF. The score and reveal sounds are generated in code; they do not sample a third-party music recording. The fictional scenarios and diagrams illustrate the shared Markdown concepts. Font and narration-engine attribution is supplied in THIRD_PARTY_NOTICES.md; the accompanying Inter license covers the embedded document font.

The source concepts are attributed in the Study Guide and content/references.md to the supplied course chapters, Sutton and Barto’s Reinforcement Learning: An Introduction (2nd edition), Nand to Tetris, relevant IETF RFCs, NIST AI RMF and Unicode documentation. Those optional references supplement the explanations in these files.

Verification and required human review

Scripts and chapter references are checked against the measured lesson JSON and canonical Markdown topic map. Numerical demonstrations and practice answers agree with the supplied runnable models. Final MP4 metadata is filled from the actual exported files when available. A teammate’s full viewing of every final video and a beginner’s attempt at a selected learning check must be recorded only after they occur; no such independent human review is invented in this guide. Machine checks do not replace that requirement.

Video 01 — Basic reinforcement learning: narration script and teaching outline

Video file: BlueImpulse_Video01_Basic_Reinforcement_Learning.mp4
Viewing position: 1 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 47,425,849 bytes (47.43 MB)

Prerequisites: Basic arithmetic and the meanings of state and action. No programming experience is needed.

Learning goals:

- Name the learning loop
- Balance exploration and exploitation
- Calculate one Q-value update

Chapter and subtopic connections:

- Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop
- Chapter 2 — Behaviorism & Behaviorist AI / Rewards, policies & values
- Chapter 2 — Behaviorism & Behaviorist AI / Explore or exploit?
- Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value
- Chapter 2 — Behaviorism & Behaviorist AI / Training is different from using a policy

Apply in the learning system: Complete ACT-02 in the Study Guide / Markdown activity file. Use Test Bank questions TF05–TF08, MC05–MC08 and OE02 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic rl
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:32	Pip's route is blocked	Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop
00:32–01:11	Today's route	Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop
01:11–02:15	Follow the learning loop	Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop
02:15–03:19	Reward, policy, value	Chapter 2 — Behaviorism & Behaviorist AI / Rewards, policies & values
03:19–04:11	The restaurant analogy	Chapter 2 — Behaviorism & Behaviorist AI / Explore or exploit?
04:11–04:53	Is the familiar route certainly best?	Chapter 2 — Behaviorism & Behaviorist AI / Explore or exploit?
04:53–05:38	What is a Q value?	Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value
05:38–06:36	Work through the worked update	Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value
06:36–07:45	A better estimate, not a proof	Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value
07:45–08:29	Try a terminal state	Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value
08:29–09:30	Experience changes estimates	Chapter 2 — Behaviorism & Behaviorist AI / Training is different from using a policy
09:30–10:00	Pip completes the shift	Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:32 — Pip's route is blocked

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop

Teaching / visual direction: Keep Pip’s problem visible while the opening question establishes why the concept matters. Key visual labels: PIP / ROUTE BLOCKED / TRY AGAIN.

00:00.150–00:02.347: Pip, the coffee is getting cold!

00:02.347–00:08.533: Our simulated campus rover has a delivery to make, but its familiar route is blocked.

00:08.533–00:17.133: Repeating yesterday's move will not solve today's problem. Pip needs useful feedback, and a way to change its estimates.

00:17.133–00:21.591: Have you ever wondered how an agent learns which action to try?

00:21.591–00:30.660: In Chapter two, Behaviorism and Behaviorist AI, we will connect the reinforcement-learning loop to one numerical update.

00:32–01:11 — Today’s route

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: LOOP / CHOICES / Q UPDATE.

00:32.498–00:39.921: Today you will identify an agent, an environment, a state, an action, a reward, and a policy.

00:39.921–00:47.984: You will compare exploration with exploitation, then calculate the example in the subtopic Update a Q value.

00:47.984–00:53.338: Welcome to the tutorial on basic reinforcement learning by Blue Impulse.

00:53.338–01:01.338: Have paper ready for one calculation. The tutorial PDF includes the same worked steps and practice question.

01:01.338–01:09.468: Treat our rover as an illustration. We are explaining a learning method, not claiming every real rover learns this way.

01:11–02:15 — Follow the learning loop

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: STATE / ACTION / REWARD / NEXT STATE.

01:11.628–01:20.271: Start with the subtopic The reinforcement-learning loop. One interaction connects an observation, an action, and feedback.

01:20.271–01:27.097: The agent selects an action. The environment changes and returns the next state and a reward.

01:27.097–01:34.968: The learning algorithm uses that experience to improve the policy, which is the strategy for choosing actions.

01:34.968–01:40.536: For our grid, the state could be the current square, and an action could be moving right.

01:40.536–01:47.938: The reward is a number associated with the result. It is feedback, not a complete instruction manual.

01:47.938–01:54.102: Notice the return arrow. Learning depends on connecting an action to what happened after it.

01:54.102–02:02.617: For Pip, define the state before choosing an action. A location alone may be insufficient if a barrier can change the route.

02:02.617–02:12.220: The environment includes what happens after the action. The reward is feedback about that outcome, not a replacement for observing the next state.

02:15–03:19 — Reward, policy, value

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Rewards, policies & values

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: REWARD / POLICY / VALUE.

02:15.280–02:21.914: The subtopic Rewards, policies and values separates three ideas that are easy to confuse.

02:21.914–02:28.442: Think of a journey. The reward scores an outcome now, while the policy chooses the next move.

02:28.442–02:34.798: A value estimate predicts how much return a choice may lead to, including later outcomes.

02:34.798–02:41.326: A route with a small cost now can still be worthwhile if it reaches a better destination.

02:41.326–02:47.981: That is why choosing the biggest immediate reward is not always the best long-term decision.

02:47.981–02:54.316: A poorly chosen reward can encourage a shortcut that misses the designer's real goal.

02:54.316–03:04.708: Suppose the reward gives a point for moving, regardless of direction. Pip could keep moving in circles and collect points without completing a delivery.

03:04.708–03:16.341: An objective and a reward design are connected, but they are not automatically identical. Test whether the rewarded behaviour is the behaviour you actually wanted.

03:19–04:11 — The restaurant analogy

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Explore or exploit?

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: EXPLORE / EXPLOIT / COMPARE.

03:19.454–03:24.040: Use a restaurant to understand the subtopic Explore or exploit?

03:24.040–03:31.805: Ordering the dish you currently believe is best is exploitation. Trying a different dish is exploration.

03:31.805–03:37.479: Exploration gathers information because your current estimates might be wrong.

03:37.479–03:45.756: In our epsilon-greedy example, epsilon is zero point one, giving a ten percent exploration probability.

03:45.756–03:50.768: Otherwise, the agent chooses an action with the highest current Q value.

03:50.768–03:59.902: These percentages describe probabilities over choices, not a promise that exactly one out of every ten choices explores.

03:59.902–04:09.056: In a real physical system, exploration also needs safety constraints. A simulation is a useful place to experiment.

04:11–04:53 — Is the familiar route certainly best?

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Explore or exploit?

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: FAMILIAR / UNTRIED / EVIDENCE?.

Learner question: Two successes versus an untried route: proof?

Thinking interval: 04:23.232–04:28.232. The viewer may pause for longer.

04:11.628–04:17.452: Pause and think. One route has worked twice. Another route has never been tried.

04:17.452–04:23.232: Does that prove the first route is better? Take a few seconds before we reveal the answer.

04:28.232–04:34.354: No. The evidence is incomplete, so the familiar route is not certainly the better one.

04:34.354–04:42.396: Exploration may improve the estimate, but it has a cost. The design must balance information and performance.

04:42.396–04:50.950: Write one sentence explaining why exploitation uses estimated future return, rather than only the next reward.

Expected learning-check result: No. The evidence is incomplete.

04:53–05:38 — What is a Q value?

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: OLD ESTIMATE / TARGET / UPDATE.

04:53.367–05:01.708: Now use the subtopic Update a Q value. Q associates a state and an action with an estimated return.

05:01.708–05:09.302: Q-learning is called model-free because it does not require a supplied model of environment transitions.

05:09.302–05:16.725: Model-free does not mean memory-free. The estimates still live in a table or another representation.

05:16.725–05:22.357: The update combines an old estimate with a target derived from new experience.

05:22.357–05:30.335: Alpha controls how far the estimate moves toward that target. Gamma discounts the future-value contribution.

05:30.335–05:36.179: We will keep every input visible, so you can reproduce the arithmetic yourself.

05:38–06:36 — Work through the worked update

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: OLD Q = 2 / TARGET = 4.6 / NEW Q = 3.3.

05:38.759–05:45.051: Start with an old Q value of two and a learning rate, alpha, of zero point five.

05:45.051–05:51.131: The reward is one, gamma is zero point nine, and the best next Q value is four.

05:51.131–05:58.511: First calculate the target: one plus zero point nine times four. The target is four point six.

05:58.511–06:05.615: Next subtract the old estimate. Four point six minus two gives an error of two point six.

06:05.615–06:11.310: Multiply that error by alpha. Half of two point six is one point three.

06:11.310–06:17.795: Finally add the change to the old value. Two plus one point three gives three point three.

06:17.795–06:25.819: Watch the estimate move halfway toward the target. That halfway movement is what alpha equals one half means here.

06:25.819–06:33.669: The supporting Python file repeats this calculation and prints every intermediate value for you to inspect.

06:36–07:45 — A better estimate, not a proof

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ONE EXPERIENCE / BETTER ESTIMATE / NO GUARANTEE.

06:36.672–06:44.436: This experience raised the estimate from two to three point three. Now decide what that result actually means.

06:44.436–06:51.668: It did not prove that the action is always best. One observation is not a reliable policy.

06:51.668–06:58.408: Repeated experience, an appropriate learning schedule, and enough exploration are still important.

06:58.408–07:05.021: There is also a special case: when the next state is terminal, the future-value term is zero.

07:05.021–07:11.186: That prevents the calculation from inventing future reward after the episode has ended.

07:11.186–07:19.078: Compare these two ideas: updating an estimate from evidence, and guaranteeing the outcome of the next action.

07:19.078–07:24.048: They are different claims, and only the first follows from our calculation.

07:24.048–07:33.139: Imagine that the next trial gives different feedback. One sample is noisy evidence, so another update can move the estimate again.

07:33.139–07:42.592: The learning rate controls how strongly this experience changes the estimate. It does not turn a single successful delivery into a guarantee.

07:45–08:29 — Try a terminal state

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Update a Q value

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: TERMINAL STATE / NO FUTURE RETURN / NEW Q = 1.5.

Learner question: Terminal state: old Q = 2, alpha = 0.5, reward = 1

Thinking interval: 07:58.414–08:04.414. The viewer may pause for longer.

07:46.063–07:51.908: Your turn. Keep the old value at two, alpha at one half, and the reward at one.

07:51.908–07:58.414: Make the next state terminal. What is the new Q value? Pause the video if you need more time.

08:04.414–08:12.114: The future contribution is zero, so the target is one. The error is one minus two, or negative one.

08:12.114–08:19.218: Half that error is negative one half. Add it to two, giving a new value of one point five.

08:19.218–08:27.711: The estimate falls because the observed target is lower. A learning update can decrease a value as well as increase it.

Expected learning-check result: New Q = 2 + 0.5 × (1 − 2) = 1.5

08:29–09:30 — Experience changes estimates

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / Training is different from using a policy

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: OBSERVE / ACT / LEARN / TRY.

08:29.889–08:37.312: Return to Pip's blocked route. The agent acts, receives feedback, and learns a policy from experience.

08:37.312–08:47.790: Reward scores an outcome. Value estimates return. Exploration improves information, while exploitation uses the best current estimates.

08:47.790–08:55.021: Our worked Q update produced three point three, and our terminal-state practice produced one point five.

08:55.021–09:03.216: The training-and-deployment distinction matters: running an already learned policy is not the same as training it online.

09:03.216–09:08.335: A fixed sensor-response rule can control a machine without learning at all.

09:08.335–09:16.462: Before moving on, explain the loop to someone else and run the included calculation with one changed reward.

09:16.462–09:27.686: Pip's improvement is a change in an estimate and, possibly, a later choice. It is not proof that every new street or unexpected obstacle has been solved.

09:30–10:00 — Pip completes the shift

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / The reinforcement-learning loop

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:30.933–09:34.857: Pip's small problem now has an explanation you can test.

09:34.857–09:42.750: Use Chapter 2, Behaviorism & Behaviorist AI, and the named subtopics to revisit the reasoning.

09:42.750–09:52.203: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:52.203–09:57.707: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

Use old Q = 2, alpha = 0.5, reward = 1, gamma = 0.9 and best next Q = 4. Alpha is the learning rate and gamma discounts future return. The step-by-step arithmetic is:

Step	Calculation	Result
Target	1 + 0.9 × 4	4.6
Error	4.6 − 2	2.6
Change	0.5 × 2.6	1.3
Updated value	2 + 1.3	3.3

Practice: Keep old Q, alpha and reward unchanged but make the next state terminal. Predict the new value before reading on.

Checked answer: The future contribution is zero. Target = 1; error = −1; change = −0.5; new Q = 1.5. An estimate may fall after evidence as well as rise. Neither update proves that the next action will succeed.

Video 02 — How hardware enables AI: narration script and teaching outline

Video file: BlueImpulse_Video02_How_Hardware_Enables_AI.mp4
Viewing position: 2 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 48,161,007 bytes (48.16 MB)

Prerequisites: Know that software executes instructions and stores data. Chapter 1 — The Fundamentals supplies this foundation; Video 05 is an optional preparatory lesson.

Learning goals:

- Trace sensing to action
- Explain memory and processing roles
- Diagnose a hardware bottleneck

Chapter and subtopic connections:

- Chapter 3 — Hardware / Hardware gives software a body
- Chapter 3 — Hardware / Memory: capacity versus speed
- Chapter 3 — Hardware / The hardware behind an AI model
- Chapter 3 — Hardware / Overlap work with a pipeline
- Chapter 3 — Hardware / Sense, decide, act, sense again
- Chapter 6 — Strengths & Limitations / Four physical budgets

Apply in the learning system: Complete ACT-03 in the Study Guide / Markdown activity file. Use Test Bank questions TF09–TF12, MC09–MC12 and OE03 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic hardware
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:28	Pip’s plan is waiting	Chapter 3 — Hardware / Hardware gives software a body
00:28–01:04	Today’s route	Chapter 3 — Hardware / Hardware gives software a body
01:04–01:52	Follow the physical path	Chapter 3 — Hardware / Hardware gives software a body
01:52–02:35	Use the kitchen analogy	Chapter 3 — Hardware / Memory: capacity versus speed
02:35–03:47	The processor may be waiting	Chapter 3 — Hardware / Memory: capacity versus speed
03:47–04:39	Match the processor to the work	Chapter 3 — Hardware / The hardware behind an AI model
04:39–05:45	Overlap work, increase throughput	Chapter 3 — Hardware / Overlap work with a pipeline
05:45–06:49	A signal is not motor power	Chapter 3 — Hardware / Sense, decide, act, sense again
06:49–07:35	Close the loop	Chapter 3 — Hardware / Sense, decide, act, sense again
07:35–08:32	Will a faster GPU fix this?	Chapter 6 — Strengths & Limitations / Four physical budgets
08:32–09:32	Hardware enables every step	Chapter 3 — Hardware / The hardware behind an AI model
09:32–10:00	Pip completes the shift	Chapter 3 — Hardware / Hardware gives software a body

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:28 — Pip’s plan is waiting

Content reference: Chapter 3 — Hardware / Hardware gives software a body

Teaching / visual direction: Keep Pip’s problem visible while the opening question establishes why the concept matters. Key visual labels: PLAN READY / DATA WAITING / WHEELS WAIT.

00:00.150–00:02.965: Pip has a plan. The wheels still wait.

00:02.965–00:08.703: A sensor stream reaches our campus rover faster than its computer can process it.

00:08.703–00:15.743: A clever decision still needs memory, processing, connections, and enough power to become an action.

00:15.743–00:20.542: Have you ever wondered what gives an AI decision a physical body?

00:20.542–00:26.707: Chapter three, Hardware, will help us follow the whole path and find the actual bottleneck.

00:28–01:04 — Today’s route

Content reference: Chapter 3 — Hardware / Hardware gives software a body

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: HARDWARE PATH / MEMORY / PIPELINE.

00:28.846–00:35.245: You will trace the sensing, memory, processing, and actuation path of a small robot.

00:35.245–00:42.455: You will explain why memory movement can limit a fast processor and what a pipeline actually improves.

00:42.455–00:49.622: You will also choose hardware by its job, rather than assuming every AI system needs the same chip.

00:49.622–00:54.998: Welcome to the tutorial on how hardware enables AI by Blue Impulse.

00:54.998–01:01.482: Follow each part of the diagram, then predict what happens when that part becomes the bottleneck.

01:04–01:52 — Follow the physical path

Content reference: Chapter 3 — Hardware / Hardware gives software a body

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: INPUT / PROCESS / OUTPUT / STORE.

01:04.324–01:11.897: The subtopic Hardware gives software a body groups hardware by input, processing, output, and storage.

01:11.897–01:20.195: For our robot, the sensor supplies observations. Memory holds inputs, model parameters, and working results.

01:20.195–01:27.127: The processor evaluates the program or model, and an output communicates the resulting decision.

01:27.127–01:33.271: If the robot needs to move, an actuator must turn that decision into physical motion.

01:33.271–01:40.929: Notice that one device can have multiple roles. A touchscreen both displays output and senses input.

01:40.929–01:48.544: The diagram is a chain of responsibilities, not a claim that every box must be a separate physical chip.

01:52–02:35 — Use the kitchen analogy

Content reference: Chapter 3 — Hardware / Memory: capacity versus speed

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: CACHE / RAM / STORAGE.

01:52.324–01:56.121: Use a kitchen to remember the memory hierarchy.

01:56.121–02:03.419: Registers are like the tiny workspace immediately in your hands. Cache keeps useful ingredients close to the cook.

02:03.419–02:10.352: Main memory is the larger active workbench, while persistent storage is more like the pantry.

02:10.352–02:16.495: The analogy is about access and capacity, not a literal map of computer components.

02:16.495–02:24.601: A huge pantry does not help if ingredients arrive too slowly. Similarly, a processor can wait for data.

02:24.601–02:32.579: Capacity asks how much fits. Memory bandwidth asks how quickly data can move. These are different questions.

02:35–03:47 — The processor may be waiting

Content reference: Chapter 3 — Hardware / Memory: capacity versus speed

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: WEIGHTS / INPUTS / INTERMEDIATE RESULTS.

02:35.628–02:43.567: Model weights, inputs, and intermediate results must fit somewhere. A processor cannot use data that has not arrived.

02:43.567–02:51.655: When memory bandwidth is insufficient, a fast processor may spend time waiting instead of doing useful arithmetic.

02:51.655–02:58.545: Adding storage capacity alone does not automatically remove a memory-transfer bottleneck.

02:58.545–03:07.977: The table also distinguishes cache, main RAM, and secondary storage. They are not interchangeable names for one resource.

03:07.977–03:15.998: Both static RAM and dynamic RAM are volatile, although their circuits and refresh requirements differ.

03:15.998–03:23.915: For this tutorial, focus on the system question: is the limiting factor how much fits, or how quickly it arrives?

03:23.915–03:34.051: Imagine the processor as a cook waiting for ingredients. Hiring a faster cook does not shorten the time a missing ingredient spends in transit.

03:34.051–03:42.438: Likewise, a faster arithmetic unit can still spend time waiting for memory. Measure that waiting before choosing the upgrade.

03:47–04:39 — Match the processor to the work

Content reference: Chapter 3 — Hardware / The hardware behind an AI model

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: CPU / GPU / ACCELERATOR.

03:47.628–03:52.556: Now compare the processor roles in the subtopic Hardware for AI.

03:52.556–04:02.692: A CPU handles general control, branches, and diverse tasks. A GPU can accelerate suitable parallel arithmetic workloads.

04:02.692–04:11.484: Specialised accelerators can efficiently perform supported tensor operations, but their benefits depend on the actual workload.

04:11.484–04:20.934: Simple tabular reinforcement learning can run on an ordinary CPU. AI does not always require a GPU.

04:20.934–04:27.632: Training changes model parameters. Inference uses an existing model to produce an output.

04:27.632–04:35.610: A model can be trained on a large server and deployed on a smaller device if the deployment requirements fit.

04:39–05:45 — Overlap work, increase throughput

Content reference: Chapter 3 — Hardware / Overlap work with a pipeline

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: FETCH / DECODE / EXECUTE / WRITE.

04:39.280–04:45.680: Use laundry to understand pipelining. One load can dry while a later load is washing.

04:45.680–04:51.503: In our simplified processor, the stages are fetch, decode, execute, and write.

04:51.503–04:59.460: Instruction one enters fetch in cycle one. In cycle two, it decodes while instruction two enters fetch.

04:59.460–05:08.081: Keep stepping forward. The instructions occupy different stages, so several instructions make progress during the same cycle.

05:08.081–05:15.995: After the pipeline fills, this ideal example completes one instruction per cycle, assuming no stalls.

05:15.995–05:23.120: It does not make an individual four-stage instruction take just one cycle from start to finish.

05:23.120–05:32.850: Real pipelines can pause for dependencies, uncertain branches, or shared resources. These hazards limit the ideal improvement.

05:32.850–05:40.530: Use the included Python table to compare completion cycles rather than relying only on the moving picture.

05:45–06:49 — A signal is not motor power

Content reference: Chapter 3 — Hardware / Sense, decide, act, sense again

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SENSOR / CONVERSION / CONTROL / ACTUATOR.

05:45.541–05:55.251: Trace the environmental control loop: measurement becomes a signal, processing chooses an output, and an actuator changes the environment.

05:55.251–06:03.677: An analogue sensor may need an analogue-to-digital converter. Many sensors already provide digital measurements.

06:03.677–06:10.908: The processor evaluates a decision, then a driver or power stage supplies what the actuator needs.

06:10.908–06:20.319: Some actuators use a digital interface or pulse-width modulation. They do not all require a digital-to-analogue converter.

06:20.319–06:28.681: Pulse-width modulation is timed switching, not the same thing as converting a value into an analogue voltage.

06:28.681–06:35.549: A control signal can tell a motor what to do without being able to supply the current that moves it.

06:35.549–06:45.088: Pip's control signal communicates what the motor driver should do. The energy that turns a loaded wheel still comes through the power system.

06:49–07:35 — Close the loop

Content reference: Chapter 3 — Hardware / Sense, decide, act, sense again

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: MEASURE / COMPARE / CHANGE / REPEAT.

06:49.715–06:57.032: Imagine the robot detects an obstacle, selects a turn, and sends a command to its motor driver.

06:57.032–07:03.645: The driver powers the motor, the wheels move, and the sensor measures the changed situation.

07:03.645–07:10.705: That final measurement closes the feedback loop. The command alone does not prove the movement succeeded.

07:10.705–07:17.361: A wheel might slip, a sensor might be noisy, or the actuator might reach a mechanical limit.

07:17.361–07:24.955: Monitoring and control differ. Displaying a measurement monitors; changing the environment adds control.

07:24.955–07:32.058: This loop can use a fixed rule or a learned policy. The hardware path alone does not tell you which.

07:35–08:32 — Will a faster GPU fix this?

Content reference: Chapter 6 — Strengths & Limitations / Four physical budgets

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: FASTER GPU? / MEMORY WAIT / MEASURE FIRST.

Learner question: Waiting for data + insufficient motor current

Thinking interval: 07:52.057–07:58.057. The viewer may pause for longer.

07:35.628–07:44.442: Pause and diagnose a system. Its processor repeatedly waits for data, and its motor driver cannot supply enough current.

07:44.442–07:52.057: Would replacing the processor with a faster GPU solve both problems? Think before the answer appears.

07:58.057–08:06.227: No. Memory movement and actuator power are separate limits that a new GPU does not automatically remove.

08:06.227–08:12.541: Measure the path, identify the limiting stage, and match the intervention to that evidence.

08:12.541–08:18.898: Chapter six, Strengths and Limitations, adds power, heat, memory, and physical constraints.

08:18.898–08:28.116: An advertised arithmetic rate describes one capability. It is not a complete measurement of the robot's useful performance.

Expected learning-check result: Fix the memory path and power stage.

08:32–09:32 — Hardware enables every step

Content reference: Chapter 3 — Hardware / The hardware behind an AI model

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: DATA / COMPUTE / POWER / ACTION.

08:32.498–08:42.122: Trace Pip's whole path: observations enter, memory holds information, processing evaluates it, and outputs communicate or act.

08:42.122–08:49.950: Memory capacity and bandwidth both matter. Pipeline throughput is different from the latency of one instruction.

08:49.950–08:57.352: Conversion, control signals, actuator power, and processor strengths all affect what the system can do.

08:57.352–09:02.706: Choose hardware by the workload, deadlines, power budget, and physical task.

09:02.706–09:10.858: To check your understanding, draw a robot's path from camera to wheel, and label one possible bottleneck at each stage.

09:10.858–09:17.002: Then explain why a more powerful chip does not, by itself, create a learning algorithm.

09:17.002–09:27.970: An upgrade should target the constrained part of the task. Check the input rate, memory traffic, compute load, and actuator response as a connected system.

09:32–10:00 — Pip completes the shift

Content reference: Chapter 3 — Hardware / Hardware gives software a body

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:32.498–09:36.422: Pip's small problem now has an explanation you can test.

09:36.422–09:42.011: Use Chapter 3, Hardware, and the named subtopics to revisit the reasoning.

09:42.011–09:51.464: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:51.464–09:56.968: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

Assume four equal one-cycle stages: F = fetch, D = decode, E = execute, W = write-back. Instructions are independent and no stage stalls. Record the occupied stage in each cycle:

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6
I1	F	D	E	W	—	—
I2	—	F	D	E	W	—
I3	—	—	F	D	E	W

Completion cycles are 4, 5 and 6. Three instructions finish in six cycles, rather than twelve sequential stage cycles. Each instruction still traverses four stages: pipeline throughput and individual latency are different.

Practice: The processor repeatedly waits for data, and a motor driver cannot supply enough current. Does a faster GPU fix both problems?

Checked answer: No. Memory movement and actuator power are distinct bottlenecks. Measure the limiting path, improve the memory transfer or working set as appropriate, and size the driver/power system for the actuator. A processor upgrade alone does not establish either fix.

Video 03 — Network challenges: narration script and teaching outline

Video file: BlueImpulse_Video03_Network_Challenges.mp4
Viewing position: 3 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 48,902,815 bytes (48.90 MB)

Prerequisites: Be comfortable with multiplication and converting milliseconds to seconds. Chapter 4 — Networking introduces packets and protocol layers.

Learning goals:

- Distinguish five network measures
- Calculate the effect of delay
- Choose a response to the actual problem

Chapter and subtopic connections:

- Chapter 4 — Networking / Capacity is not responsiveness
- Chapter 4 — Networking / Sharing a communication medium
- Chapter 4 — Networking / Give each protocol layer a job
- Chapter 5 — Applications / Real-time means meeting a deadline

Apply in the learning system: Complete ACT-04 in the Study Guide / Markdown activity file. Use Test Bank questions TF13–TF16, MC13–MC16 and OE04 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic network
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:30	Pip's message is late	Chapter 4 — Networking / Capacity is not responsiveness
00:30–01:06	Today's route	Chapter 4 — Networking / Capacity is not responsiveness
01:06–01:51	Name the problem before fixing it	Chapter 4 — Networking / Capacity is not responsiveness
01:51–02:50	Use the road analogy	Chapter 4 — Networking / Capacity is not responsiveness
02:50–03:36	Shared demand changes the result	Chapter 4 — Networking / Sharing a communication medium
03:36–04:28	Reliability has a timing cost	Chapter 4 — Networking / Give each protocol layer a job
04:28–05:37	What happens during the delay?	Chapter 5 — Applications / Real-time means meeting a deadline
05:37–06:42	Real-time means a deadline	Chapter 5 — Applications / Real-time means meeting a deadline
06:42–07:37	Match the remedy to the symptom	Chapter 4 — Networking / Capacity is not responsiveness
07:37–08:31	Calculate a new delay	Chapter 5 — Applications / Real-time means meeting a deadline
08:31–09:31	Measure the whole task	Chapter 4 — Networking / Capacity is not responsiveness
09:31–10:00	Pip completes the shift	Chapter 4 — Networking / Capacity is not responsiveness

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:30 — Pip's message is late

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Keep Pip's problem visible while the opening question establishes why the concept matters. Key visual labels: MESSAGE SENT / MESSAGE LATE / WHY?.

00:00.150–00:03.541: The message says stop. Pip has already moved.

00:03.541–00:11.071: Our simulated delivery rover received a message that was useful when it was sent, but late when it arrived.

00:11.071–00:16.873: A strong Wi-Fi signal does not tell us how quickly the whole task will finish.

00:16.873–00:21.289: Have you ever wondered why a fast connection can still feel slow?

00:21.289–00:28.392: Chapter four, Networking, gives us the measurements. Chapter five, Applications, connects them to deadlines.

00:30–01:06 — Today's route

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: CAPACITY / DELAY / DEADLINES.

00:30.933–00:36.457: Today you will distinguish bandwidth, throughput, latency, jitter, and packet loss.

00:36.457–00:42.153: You will use a numerical example to connect delay with a physical consequence.

00:42.153–00:50.475: You will choose responses such as compression, fewer round trips, buffering, or local control for specific problems.

00:50.475–00:55.168: Welcome to the tutorial on network challenges by Blue Impulse.

00:55.168–01:03.747: Keep a calculator or paper ready. Try the delay calculation before checking the worked answer in the tutorial PDF.

01:06–01:51 — Name the problem before fixing it

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: BANDWIDTH / THROUGHPUT / LATENCY / JITTER / LOSS.

01:06.411–01:14.560: In the subtopic Capacity is not responsiveness, bandwidth describes how much data a link can carry per second.

01:14.560–01:21.343: Throughput is the successful transfer rate actually achieved. It can be lower than the link's capacity.

01:21.343–01:27.913: Latency is how long a transfer or response takes. Jitter is variation in that delay.

01:27.913–01:31.944: Packet loss means some packets do not arrive as expected.

01:31.944–01:40.054: These measures answer different questions. A connection can carry a lot of data and still take a long time to respond.

01:40.054–01:48.504: When describing an experiment, specify what you measured: one-way delay, round-trip delay, or complete task response time.

01:51–02:50 — Use the road analogy

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ROAD WIDTH / TRAVEL TIME / TRAFFIC.

01:51.802–01:58.159: Imagine a road. A wider road can carry more cars at once, like greater bandwidth.

01:58.159–02:02.724: How many cars actually arrive each second is closer to throughput.

02:02.724–02:12.177: How long one journey takes is closer to latency. Uneven arrival times illustrate jitter, while missing deliveries illustrate loss.

02:12.177–02:18.299: Widening a road does not shorten the distance to the destination or remove every queue.

02:18.299–02:28.136: Likewise, increasing link capacity does not erase propagation time, server processing, or unnecessary back-and-forth exchanges.

02:28.136–02:35.517: The analogy helps separate the ideas, but measure the actual network before deciding which factor dominates.

02:35.517–02:46.698: A road can be wide and still contain a long traffic-light delay. That is why a capacity figure cannot by itself predict a small message's arrival time.

02:50–03:36 — Shared demand changes the result

Content reference: Chapter 4 — Networking / Sharing a communication medium

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SHARED MEDIUM / COMPETITION / QUEUE.

02:50.759–02:55.451: Now connect performance metrics to transmission media and contention.

02:55.451–03:02.683: Wireless devices may share a medium, compete for transmission time, and experience interference.

03:02.683–03:10.127: Heavy demand can create queues. Retransmissions after loss can use more capacity and add delay.

03:10.127–03:17.273: Modern switched full-duplex Ethernet should not be described as suffering the old shared-hub collision model.

03:17.273–03:24.099: The useful diagnosis depends on the network technology, the traffic, and the end-to-end path.

03:24.099–03:32.934: Try testing near the access point, with fewer competing transfers, and at different times before blaming one advertised number.

03:36–04:28 — Reliability has a timing cost

Content reference: Chapter 4 — Networking / Give each protocol layer a job

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels:

IP: ROUTING / TCP: DELIVERY / TIME STILL PASSES.

03:36.150–03:45.579: In the TCP and IP subtopic, IP routes packets, while TCP can provide an ordered, reliable byte stream.

03:45.579–03:52.021: Reliability can require acknowledgements and retransmission, so recovery may add delay.

03:52.021–04:00.258: A transport choice does not by itself guarantee an application deadline or protect every message from an attacker.

04:00.258–04:09.391: Some applications prefer timely delivery and handle missing data themselves. Others need complete, correctly ordered transfer.

04:09.391–04:16.367: Read the application's requirements before selecting a protocol or interpreting a performance measurement.

04:16.367–04:25.049: The security subtopics distinguish confidentiality, integrity, and availability from simply having a fast link.

04:28–05:37 — What happens during the delay?

Content reference: Chapter 5 — Applications / Real-time means meeting a deadline

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SPEED = 30 m/s / TIME IN SECONDS / DISTANCE = SPEED x TIME.

04:28.846–04:38.491: For this calculation, switch to an illustrative vehicle travelling at a constant thirty metres per second. This is not Pip's campus speed.

04:38.491–04:46.384: Distance travelled during delay equals speed multiplied by delay. First convert milliseconds into seconds.

04:46.384–04:54.788: Five hundred milliseconds is zero point five seconds. Thirty times zero point five gives fifteen metres.

04:54.788–05:01.593: Fifty milliseconds is zero point zero five seconds. That gives one point five metres.

05:01.593–05:09.315: Five milliseconds is zero point zero zero five seconds. That gives zero point one five metres.

05:09.315–05:16.568: Watch the three distances grow from the same starting point. We changed delay while keeping speed fixed.

05:16.568–05:24.375: These are distances before a response begins, not complete stopping distances and not safety guarantees.

05:24.375–05:32.911: Sensing, processing, actuation, and physical braking can contribute additional delay or distance in a real system.

05:37–06:42 — Real-time means a deadline

Content reference: Chapter 5 — Applications / Real-time means meeting a deadline

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: DEFINED START / DEADLINE / DEFINED FINISH.

05:37.715–05:46.547: Chapter five, Applications, explains real-time response. A system must respond within a specified deadline.

05:46.547–05:54.823: Real-time does not simply mean very fast. Correctness includes whether the result arrives in time for its task.

05:54.823–06:02.954: Missing a hard deadline can violate the system's requirements. Missing a soft deadline reduces quality of service.

06:02.954–06:11.341: Moving a computation locally may remove some communication delay, but does not automatically guarantee a safe outcome.

06:11.341–06:19.983: Use this distinction when evaluating a remote controller: what is its deadline, and what happens if the response is late?

06:19.983–06:26.489: A good average is insufficient when the important problem is an occasional deadline miss.

06:26.489–06:37.649: A deadline needs a defined start and finish. State whether you measured just transmission, or sensing, queueing, computation, and the final response as well.

06:42–07:37 — Match the remedy to the symptom

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: MEASURE / MATCH / CHANGE / CHECK.

06:42.411–06:48.960: Match each response to a measured problem instead of choosing the fastest-looking specification.

06:48.960–06:58.413: When capacity is limited, compress or summarise suitable data. When round trips dominate, reduce unnecessary exchanges.

06:58.413–07:06.007: Timestamps help detect stale measurements. A playback buffer can smooth jitter, but the buffer adds delay.

07:06.007–07:16.122: Reconnect logic and redundancy can improve resilience, while designed local control reduces dependence on an unavailable remote link.

07:16.122–07:25.618: Chapter six, Strengths and Limitations, reminds us that growth adds load and coordination challenges, not just more devices.

07:25.618–07:33.638: Every remedy has a trade-off. Diagnose, change one factor, and measure again using the metric that matters.

07:37–08:31 — Calculate a new delay

Content reference: Chapter 5 — Applications / Real-time means meeting a deadline

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: 200 ms / 0.2 s / 6 m.

Learner question: 30 m/s × 200 ms = ?

Thinking interval: 07:52.242–07:58.242. The viewer may pause for longer.

07:37.715–07:45.608: Your turn. At thirty metres per second, how far does an object travel during a two hundred millisecond delay?

07:45.608–07:52.242: Convert the units before multiplying. Take a few seconds, or pause the video to work it out.

07:58.242–08:05.537: Two hundred milliseconds is zero point two seconds. Thirty times zero point two is six metres.

08:05.537–08:12.342: This is only distance during the delay. It does not include the rest of a real stopping process.

08:12.342–08:22.243: Now consider a separate question: would adding a playback buffer always help a time-critical controller? No, because buffering adds delay.

08:22.243–08:28.216: The right network design depends on the task, not a single universally best setting.

Expected learning-check result: 30 × 0.200 = 6 metres during delay

08:31–09:31 — Measure the whole task

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: CAPACITY / RESPONSIVENESS / RELIABILITY.

08:31.976–08:40.896: Capacity is the limit, throughput is achieved transfer, latency is delay, jitter is variation, and loss is missing data.

08:40.896–08:49.151: Our demonstration gave fifteen, one point five, and zero point one five metres during three different delays.

08:49.151–08:54.228: Those numbers illustrate timing; they are not total stopping distances.

08:54.228–09:01.353: Use the road analogy to keep the measures distinct, then replace the analogy with an actual measurement.

09:01.353–09:09.971: For practice, describe a slow application, name the metric you would test, and justify one targeted improvement.

09:09.971–09:16.477: Run the supporting delay calculator to check a second speed and explain the units in your answer.

09:16.477–09:27.082: Pip's late message is a timing problem first. More capacity may help if a queue is the cause, but a measurement must connect the proposed change to that cause.

09:31–10:00 — Pip completes the shift

Content reference: Chapter 4 — Networking / Capacity is not responsiveness

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:31.454–09:35.379: Pip's small problem now has an explanation you can test.

09:35.379–09:41.330: Use Chapter 4, Networking, and the named subtopics to revisit the reasoning.

09:41.330–09:50.784: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:50.784–09:56.287: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

Use an illustrative vehicle moving at 30 m/s. This is not Pip’s campus speed. Convert milliseconds to seconds before multiplying:

Communication delay	Seconds	Distance during delay
500 ms	0.500 s	15 m
50 ms	0.050 s	1.5 m
5 ms	0.005 s	0.15 m

Distance during delay = speed × delay. This is not total stopping distance: sensing, later computation, actuator response and braking contribute additional terms.

Practice: How far is travelled during 200 ms at the same speed? Would a playback buffer always help a time-critical controller?

Checked answer: 200 ms = 0.200 s, so 30 × 0.200 = 6 m. A playback buffer can smooth jitter but adds delay. A controller’s useful design depends on its deadline and failure response, not a universal “more buffering is better” rule.

Video 04 — Networking for behaviorist AI: narration script and teaching outline

Video file: BlueImpulse_Video04_Networking_for_Behaviorist_AI.mp4
Viewing position: 4 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 50,061,875 bytes (50.06 MB)

Prerequisites: Understand fixed reactive rules versus learned policies (Video 01 / Chapter 2), and latency versus bandwidth (Video 03 / Chapter 4).

Learning goals:

- Separate local control from shared services
- Interpret network messages safely
- Associate feedback with its original action

Chapter and subtopic connections:

- Chapter 5 — Applications / Networking for a behaviour-based agent
- Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots
- Chapter 6 — Strengths & Limitations / Put each computation where it fits

Apply in the learning system: Complete ACT-05 in the Study Guide / Markdown activity file. Use Test Bank questions TF17–TF20, MC17–MC20 and OE05 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic networked-ai
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:29	Three commands. One decision.	Chapter 5 — Applications / Networking for a behaviour-based agent
00:29–01:06	Today's route	Chapter 5 — Applications / Networking for a behaviour-based agent
01:06–01:59	Reactive behaviour is not always learning	Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots
01:59–02:45	Keep the time-critical loop close	Chapter 5 — Applications / Networking for a behaviour-based agent
02:45–03:36	Let the network do useful shared work	Chapter 5 — Applications / Networking for a behaviour-based agent
03:36–04:41	A number needs context	Chapter 5 — Applications / Networking for a behaviour-based agent
04:41–05:54	Inspect three incoming messages	Chapter 5 — Applications / Networking for a behaviour-based agent
05:54–06:43	Match feedback to the original action	Chapter 5 — Applications / Networking for a behaviour-based agent
06:43–07:31	Define what an outage means	Chapter 6 — Strengths & Limitations / Put each computation where it fits
07:31–08:26	The feedback packet never arrived	Chapter 5 — Applications / Networking for a behaviour-based agent
08:26–09:31	Connect the responsibilities	Chapter 5 — Applications / Networking for a behaviour-based agent
09:31–10:00	Pip completes the shift	Chapter 5 — Applications / Networking for a behaviour-based agent

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:29 — Three commands. One decision.

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Keep Pip's problem visible while the opening question establishes why the concept matters. Key visual labels: FRESH / LATE / REPEATED.

00:00.150–00:03.307: Three commands. Only one belongs now.

00:03.307–00:11.885: Pip, our simulated campus delivery rover, receives a fresh command, an old command, and a command it has already handled.

00:11.885–00:16.343: They look similar on screen. Treating them alike would be a mistake.

00:16.343–00:21.505: Have you ever wondered how a connected agent preserves the meaning of a message?

00:21.505–00:28.033: Chapter five, Applications, connects local control, shared services, and message context.

00:29–01:06 — Today's route

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: LOCAL CONTROL / SHARED SERVICES / MESSAGE CONTEXT.

00:29.889–00:37.334: You will separate a fast local control loop from shared analysis, coordination, and training services.

00:37.334–00:45.760: You will explain why messages need context such as identity, time, sequence, units, and policy version.

00:45.760–00:50.644: You will also connect delayed learning feedback to the action that produced it.

00:50.644–00:56.553: Welcome to the tutorial on networking for behaviorist AI by Blue Impulse.

00:56.553–01:04.534: Connect the subtopics Real-time response, Networking for a behaviour-based agent, and Where should the intelligence run?

01:06–01:59 — Reactive behaviour is not always learning

Content reference: Chapter 2 — Behaviorism & Behaviorist AI / React first: behaviour-based robots

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: REACTIVE RULE / LEARNED POLICY / TRAINING.

01:06.933–01:16.301: Start with Chapter two, Behaviorism and Behaviorist AI. A reactive agent may map a sensed condition directly to an action.

01:16.301–01:24.684: A fixed rule such as avoiding a detected obstacle can produce useful behaviour without learning from reward.

01:24.684–01:31.531: Reinforcement learning adds a different claim: experience is used to improve a policy.

01:31.531–01:39.897: A network connection does not turn a fixed rule into a learning algorithm, and not every connected agent learns online.

01:39.897–01:48.945: For our campus delivery system example, think of a locally evaluated policy with separate, reviewed updates from a server.

01:48.945–01:56.453: We will label architectural choices clearly instead of treating them as properties of every AI system.

01:59–02:45 — Keep the time-critical loop close

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SENSE / EVALUATE / ACT.

01:59.628–02:06.305: Time-critical sensing, policy evaluation, and actuator control can stay near the device.

02:06.305–02:14.649: Our robot observes an obstacle, evaluates its current policy, and sends a control signal to the actuator hardware.

02:14.649–02:18.766: This local path need not wait for every cloud exchange.

02:18.766–02:24.909: A real-time task must meet a specified deadline, not just be fast on average.

02:24.909–02:34.853: Local processing removes some network dependency, but it still needs adequate compute, trustworthy sensing, and suitable physical controls.

02:34.853–02:42.792: Local does not automatically mean safe. The requirements and failure behaviour still have to be designed and checked.

02:45–03:36 — Let the network do useful shared work

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ROBOT / NETWORK / SERVER.

02:46.063–02:53.401: In our illustrative campus delivery system, each robot handles immediate motion control locally.

02:53.401–03:02.040: A shared service can allocate tasks, coordinate inventory, and analyse selected telemetry from several devices.

03:02.040–03:08.952: Imagine a local driver following a route while a dispatcher coordinates deliveries across the fleet.

03:08.952–03:17.274: The driver needs immediate awareness; the dispatcher benefits from a broader view. The analogy separates responsibilities.

03:17.274–03:25.533: The network may distribute reviewed policy updates, but a received update should not automatically bypass validation.

03:25.533–03:33.426: Edge and cloud are not opposing teams. Put each part of the workload where its deadlines and resource needs fit.

03:36–04:41 — A number needs context

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: IDENTITY / TIME / SEQUENCE / UNITS / POLICY.

03:36.672–03:44.586: A message needs enough context to be interpreted correctly. A number by itself is not a complete instruction.

03:44.586–03:52.649: Device identity tells us which source or destination it concerns. A timestamp helps determine freshness.

03:52.649–04:00.926: A sequence number can expose repeated or out-of-order messages. Units prevent confusing metres with centimetres.

04:00.926–04:06.493: A policy version connects an action or update to the model that produced it.

04:06.493–04:16.245: These fields are useful evidence, but fields alone do not authenticate a sender. Security checks must support the claimed identity.

04:16.245–04:24.095: Before acting, the receiver needs a policy for rejecting stale, unexpected, or unauthorised commands.

04:24.095–04:37.093: Pip is a story name; robot-seven is the device identity used in this demonstration. A receiver must check the identity expected by the protocol, not guess from a friendly label.

04:41–05:54 — Inspect three incoming messages

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ACCEPT / STALE / DUPLICATE.

04:41.367–04:50.074: Run the supporting packet-checking example. This is an illustrative receiver design, not a universal network protocol.

04:50.074–04:56.644: Our toy receiver expects robot seven, metres as the unit, and policy version three.

04:56.644–05:04.473: For this demonstration, it rejects commands older than two hundred milliseconds and sequences already processed.

05:04.473–05:13.094: The first message has the expected context, a new sequence, and an age of fifty milliseconds. It passes these checks.

05:13.094–05:21.651: The second is six hundred milliseconds old. The receiver rejects it as stale, even if the network eventually delivered it.

05:21.651–05:30.721: The third repeats an already processed sequence. The receiver rejects it as a duplicate instead of executing the command twice.

05:30.721–05:41.667: Our example assumes authentication was checked elsewhere. It does not implement cryptographic authentication or certify a safety-critical controller.

05:41.667–05:50.246: The useful lesson is why delivery alone is insufficient: the receiver must decide whether the message is still meaningful.

05:54–06:43 — Match feedback to the original action

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ACTION 42 / ACTION 43 / FEEDBACK 43.

05:54.933–05:59.625: Delayed feedback must be associated with the action that produced it.

05:59.625–06:08.545: Suppose action forty-two turns left, and action forty-three moves forward. Their feedback packets arrive in the opposite order.

06:08.545–06:14.817: Updating whichever action happened most recently would connect evidence to the wrong decision.

06:14.817–06:23.737: Use identifiers and recorded context to match each result with its original state and action before a learning update.

06:23.737–06:31.395: If a feedback packet is lost, the evidence is missing. That does not silently mean the reward was zero.

06:31.395–06:40.315: This is how networking details can affect the data used by a learning algorithm, even when policy evaluation stays local.

06:43–07:31 — Define what an outage means

Content reference: Chapter 6 — Strengths & Limitations / Put each computation where it fits

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: LOCAL TASK / SHARED TASK / SAFE STATE.

06:43.454–06:48.702: Now inspect the outage condition for our illustrative delivery system.

06:48.702–06:56.402: The design must say whether the robot pauses, completes a limited task, or enters another defined state.

06:56.402–07:03.740: Continuing safely during a link outage cannot simply be assumed because some computation is local.

07:03.740–07:12.468: Consider expiring permissions, missing coordination, conflicting routes, and how the system reconnects after the interruption.

07:12.468–07:19.422: Record what happened locally so the shared service can reconcile state when communication returns.

07:19.422–07:28.961: The right fallback depends on the task and its hazards. Our tutorial demonstrates the reasoning rather than certifying a physical robot.

07:31–08:26 — The feedback packet never arrived

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: NO PACKET / MISSING EVIDENCE / NOT ZERO REWARD.

Learner question: Missing feedback = reward zero?

Thinking interval: 07:46.076–07:52.076. The viewer may pause for longer.

07:31.976–07:38.781: Pause and think. The robot performed action forty-two, but its reward packet never arrived.

07:38.781–07:46.076: Should the learner record a reward of zero? And should a repeated command be executed again automatically?

07:52.076–08:00.761: No to both. Missing evidence is not an observed zero, and a duplicate needs the receiver's defined handling policy.

08:00.761–08:09.059: Associate any later feedback with the original action. Decide how long to wait and how to record missing data.

08:09.059–08:15.373: Then explain why a timestamp alone does not prove a message came from an authorised sender.

08:15.373–08:23.202: Use the packet demonstration to change an age or sequence and predict which validation rule should reject it.

Expected learning-check result: No. Missing evidence is not an observed zero.

08:26–09:31 — Connect the responsibilities

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: LOCAL LOOP / VALID MESSAGES / MATCHED FEEDBACK.

08:26.759–08:34.395: Keep suitable time-critical work local, and use the network for telemetry, coordination, and reviewed updates.

08:34.395–08:44.851: Messages need interpretable context and appropriate authentication. Stale or repeated commands must not be treated as automatically valid.

08:44.851–08:53.427: Delayed learning feedback belongs to its original action. Lost evidence must not silently become zero reward.

08:53.427–09:00.082: Decide which responsibility belongs on the robot, at a nearby gateway, or in the cloud.

09:00.082–09:08.661: Finally, distinguish a fixed reactive rule, a deployed learned policy, and an algorithm currently updating that policy.

09:08.661–09:16.770: Draw the local and shared responsibilities, then explain your outage behaviour before calling the architecture complete.

09:16.770–09:27.631: A message can be perfectly formatted and still be too late, duplicated, or unauthorised. Meaning depends on the receiver's state and the surrounding checks.

09:31–10:00 — Pip completes the shift

Content reference: Chapter 5 — Applications / Networking for a behaviour-based agent

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:31.976–09:35.901: Pip's small problem now has an explanation you can test.

09:35.901–09:41.959: Use Chapter 5, Applications, and the named subtopics to revisit the reasoning.

09:41.959–09:51.412: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:51.412–09:56.915: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

The toy receiver expects device ID robot-7, units m, policy_version 3, age at most 200 ms, and a sequence greater than the last accepted sequence. Start at last accepted sequence 41. Assume authentication was checked upstream; a Boolean in this demonstration is not cryptography.

Packet	Sequence	Age	Decision	Last accepted sequence after decision
A	42	50 ms	ACCEPT	42
B	43	600 ms	REJECT STALE	42
C	42	60 ms	REJECT DUPLICATE	42

Only accepted messages advance accepted-sequence state. Delivery, correct format and authenticity do not by themselves establish that a command is fresh and suitable.

Practice: Action 42 occurred but its reward packet never arrived. Is the observed reward zero? Should a repeated command automatically execute again?

Checked answer: No to both. Missing evidence is not an observed zero. Record missing data explicitly, associate any later feedback with its original transition, and follow the defined receiver policy for duplicates. Real designs also need authenticated identity, appropriate clock/session assumptions and a defined fallback.

Video 05 — Program execution: narration script and teaching outline

Video file: BlueImpulse_Video05_Program_Execution.mp4
Viewing position: 5 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 45,852,242 bytes (45.85 MB)

Prerequisites: Basic arithmetic and the idea that memory locations have addresses. No assembly-language experience is assumed.

Learning goals:

- Trace fetch, decode, and execute
- Give each register its job
- Follow a four-instruction program

Chapter and subtopic connections:

- Chapter 1 — The Fundamentals / Fetch / decode / execute
- Chapter 1 — The Fundamentals / From source code to execution
- Chapter 1 — The Fundamentals / Trace a tiny program

Apply in the learning system: Complete ACT-01 in the Study Guide / Markdown activity file. Use Test Bank questions TF01–TF04, MC01–MC04 and OE01 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic execution
```

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:32	Where does seven go?	Chapter 1 — The Fundamentals / Fetch / decode / execute
00:32–01:11	Today's route	Chapter 1 — The Fundamentals / Fetch / decode / execute
01:11–02:00	Before the processor executes	Chapter 1 — The Fundamentals / From source code to execution
02:00–02:50	Use the recipe analogy	Chapter 1 — The Fundamentals / Fetch / decode / execute
02:50–03:54	Fetch one instruction	Chapter 1 — The Fundamentals / Fetch / decode / execute
03:54–04:41	Read the four instructions	Chapter 1 — The Fundamentals / Trace a tiny program
04:41–05:51	Single-step the worked program	Chapter 1 — The Fundamentals / Trace a tiny program
05:51–06:44	Value or address?	Chapter 1 — The Fundamentals / Trace a tiny program
06:44–07:31	The next address can change	Chapter 1 — The Fundamentals / Fetch / decode / execute
07:31–08:25	Change one instruction	Chapter 1 — The Fundamentals / Trace a tiny program
08:25–09:32	Instructions change machine state	Chapter 1 — The Fundamentals / Fetch / decode / execute
09:32–10:00	Pip completes the shift	Chapter 1 — The Fundamentals / Fetch / decode / execute

Using this script

The following narration is the exact measured cue text for the revised recording. Cue timestamps refer to the video timeline; small gaps include intentional thinking time and transition space. The voice is synthetic (Kokoro ONNX, am_michael), not a recording of a student. The full teaching outline states the purpose, visual focus and learner action alongside the words spoken.

00:00–00:32 — Where does seven go?

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Keep Pip's problem visible while the opening question establishes why the concept matters. Key visual labels: 3 / + 4 / STORE 7.

00:00.150–00:03.755: Three deliveries, plus four. Where does seven go?

00:03.755–00:09.215: Pip's tiny demonstration program must add two values and save the result.

00:09.215–00:19.394: The processor follows instruction meanings and changes stored state. It does not understand the whole program like a reader understands a story.

00:19.394–00:23.980: Have you ever wondered how a line of code becomes a visible result?

00:23.980–00:31.169: Chapter one, The Fundamentals, lets us trace the registers and follow every important state change.

00:32–01:11 — Today's route

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Reveal the learning goals in the order spoken, then retain a small progress cue. Key visual labels: FETCH / DECODE / EXECUTE.

00:33.020–00:40.080: You will explain fetch, decode, and execute, and give each teaching-model register a clear job.

00:40.080–00:46.053: You will trace four instructions and distinguish an immediate value from a memory address.

00:46.053–00:55.741: You will also connect source code, translators, and machine instructions without oversimplifying how modern language implementations work.

00:55.741–01:00.626: Welcome to the tutorial on program execution by Blue Impulse.

01:00.626–01:09.652: Have a small table ready with columns for the program counter, current instruction, accumulator, and memory location two hundred.

01:11–02:00 — Before the processor executes

Content reference: Chapter 1 — The Fundamentals / From source code to execution

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: SOURCE / TRANSLATOR / MACHINE CODE.

01:11.628–01:20.399: The subtopic From source code to execution explains that machine code contains the instructions a processor executes.

01:20.399–01:30.215: An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode.

01:30.215–01:36.081: An interpreter runs operations through a runtime, sometimes working with bytecode.

01:36.081–01:44.297: The same language can have compiled and interpreted implementations, and modern systems may combine several techniques.

01:44.297–01:50.824: Avoid the blanket claim that every interpreter translates exactly one source line at a time.

01:50.824–01:58.312: Our demonstration uses invented assembly notation so that every state change remains easy to see.

02:00–02:50 — Use the recipe analogy

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: PC / MAR / MDR / CIR / ACC.

02:00.672–02:07.775: Use a recipe to remember the register jobs. The program counter marks the next recipe line.

02:07.775–02:15.838: The memory address register selects the location. The memory data register carries the item read from memory.

02:15.838–02:24.822: The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

02:24.822–02:31.584: We abbreviate those names as P C, M A R, M D R, C I R, and A C C.

02:31.584–02:39.712: The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model.

02:39.712–02:48.141: Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

02:50–03:54 — Fetch one instruction

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: ADDRESS / INSTRUCTION / DECODE / EXECUTE.

02:50.759–02:57.777: Follow the fetch sequence. First copy the address in the program counter to the memory address register.

02:57.777–03:03.216: Read the instruction at that memory location into the memory data register.

03:03.216–03:13.267: Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

03:13.267–03:21.245: The control unit can now decode the operation and its operand, and the processor executes the required work.

03:21.245–03:29.052: In this simplified model, every instruction occupies one word, so the program counter advances by one.

03:29.052–03:39.593: Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

03:39.593–03:51.180: An address identifies a place. A value is the information stored or processed there. Keeping these roles separate makes the register trace much easier to follow.

03:54–04:41 — Read the four instructions

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: LOAD #3 / ADD #4 / STORE 200 / HALT.

03:54.411–04:02.218: In the subtopic Trace a tiny program, the first instruction sits at address one hundred: LOAD, hash three.

04:02.218–04:11.309: The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator.

04:11.309–04:19.244: Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

04:19.244–04:24.513: Address one hundred and three contains HALT, which ends this teaching program.

04:24.513–04:32.811: The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator.

04:32.811–04:39.274: Before running anything, predict what that memory location will contain when the program finishes.

04:41–05:51 — Single-step the worked program

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: PC / CIR / ACC / MEMORY[200].

04:41.889–04:50.596: Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one.

04:50.596–04:56.206: Decode and execute LOAD, hash three. The accumulator now contains three.

04:56.206–05:04.269: Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

05:04.269–05:12.378: Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet.

05:12.378–05:22.557: Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred.

05:22.557–05:30.727: The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

05:30.727–05:39.711: Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops.

05:39.711–05:47.756: The runnable Python model prints these same states and register transfers, so you can reproduce the trace yourself.

05:51–06:44 — Value or address?

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: VALUE: 7 / ADDRESS: 200 / COPY.

Learner question: After STORE 200: ACC = ? and memory[200] = ?

Thinking interval: 06:04.850–06:09.850. The viewer may pause for longer.

05:51.280–05:59.304: Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred?

05:59.304–06:04.850: Also ask yourself whether STORE, two hundred means storing the number two hundred.

06:09.850–06:18.429: The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

06:18.429–06:26.322: The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role.

06:26.322–06:33.830: This distinction prevents a common tracing mistake: confusing where data lives with the value stored there.

06:33.830–06:41.620: Use this model's instruction meanings before applying assumptions from a different programming language or processor.

Expected learning-check result: ACC = 7; memory[200] = 7

06:44–07:31 — The next address can change

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: NEXT ADDRESS / BRANCH / REQUIRED EFFECT.

06:44.498–06:50.214: A branch instruction may replace the normal next value of the program counter.

06:50.214–06:57.915: That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward.

06:57.915–07:04.890: Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

07:04.890–07:14.472: Chapter three, Hardware, adds pipelining: real processors may overlap instructions while preserving the required program behaviour.

07:14.472–07:21.106: Overlapping the implementation does not make the logical effects of a program arbitrary.

07:21.106–07:29.556: For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

07:31–08:25 — Change one instruction

Content reference: Chapter 1 — The Fundamentals / Trace a tiny program

Teaching / visual direction: Keep the supplied question legible during the thinking interval; reveal the answer only after the pause. Key visual labels: LOAD #3 / ADD #9 / STORE 200.

Learner question: LOAD #3; ADD #9; STORE 200

Thinking interval: 07:46.161–07:52.161. The viewer may pause for longer.

07:31.976–07:39.485: Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred.

07:39.485–07:46.161: What value should reach memory location two hundred? Work through the accumulator before we reveal it.

07:52.161–07:59.734: LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

07:59.734–08:05.835: That is a small example of testing a prediction by changing one input to the program.

08:05.835–08:15.331: The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic.

08:15.331–08:22.733: Try another value and explain each register's role aloud as the instruction moves through the cycle.

Expected learning-check result: memory[200] = 3 + 9 = 12

08:25–09:32 — Instructions change machine state

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Follow the spoken explanation with concise labels and the corresponding diagram state; transform the existing objects into the next scene. Key visual labels: 3 + 4 / ACC = 7 / MEMORY[200] = 7.

08:25.715–08:34.465: Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation.

08:34.465–08:44.899: The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation.

08:44.899–08:55.163: The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

08:55.163–09:02.437: Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred.

09:02.437–09:09.434: Addresses and values have different roles. Branches can change the normal next instruction address.

09:09.434–09:16.879: Complete the trace without watching the highlight. Understanding means predicting each state change yourself.

09:16.879–09:28.895: Pip's seven is now a stored result because STORE copied the accumulator value to the chosen memory address. The instruction determined both what changed and what stayed the same.

09:32–10:00 — Pip completes the shift

Content reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Teaching / visual direction: Resolve the opening problem and use a concise Blue Impulse end title, returning to the opening composition. Key visual labels: BLUE IMPULSE / FOLLOW THE SIGNAL.

09:32.498–09:36.422: Pip's small problem now has an explanation you can test.

09:36.422–09:42.395: Use Chapter 1, The Fundamentals, and the named subtopics to revisit the reasoning.

09:42.395–09:51.848: The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

09:51.848–09:57.352: Blue Impulse. Follow the signal, predict the result, and check your explanation.

Supporting demonstration and limits

This invented one-word ISA is a teaching model. The # sign marks a literal value; STORE's operand is a destination address. Begin at PC 100, ACC 0 and memory[200] 0.

Fetched address	Instruction	PC after fetch	ACC after execution	memory[200]
100	LOAD #3	101	3	0
101	ADD #4	102	7	0
102	STORE 200	103	7	7
103	HALT	104	7	7

STORE copies the accumulator's 7 to location 200; it does not store 200 or change ACC to 200. Real instruction sizes, register names and fetch timing depend on the architecture.

Practice: Replace ADD #4 with ADD #9. Predict memory[200] and explain the intermediate accumulator value.

Checked answer: LOAD makes ACC 3; ADD makes ACC 12; STORE copies 12 to memory[200]. The final PC remains 104 in this model. Changing one input and predicting every state is a stronger check than copying a highlighted answer.